

Assignment 1:

The Flatland Challenge

Name: Rishabh Ray
Student ID: 34525416

Generative AI Use Statement

Generative AI tools (ChatGPT) were used as an intellectual sparring partner and writing assistant in the production of this report. Specific uses included:

- Refining the report's high-level structure and narrative flow.
- Drafting initial summaries of algorithmic analysis for concepts like time complexity and optimality, which were then manually verified, corrected, and expanded.
- Suggesting alternative formatting for tables and figures to better visualise experimental results.

The final content, including all algorithmic implementations, experimental results, and critical analysis, has been reviewed and authored by me to ensure its accuracy, originality,

Executive Summary

This report documents the design, implementation, and critical evaluation of algorithms to solve three progressively complex pathfinding tasks within the Flatland railway simulation environment. The project demonstrates an iterative approach to tackling Multi-Agent Pathfinding (MAPF), highlighting the fundamental trade-offs between solution optimality, computational scalability, and robustness to dynamic events.

- **Q1: Single-Agent Pathfinding:** Solved using a standard A star search with a Manhattan distance heuristic. The implementation was proven to be optimal and complete for the single-agent case, achieving a 100% success rate with negligible planning time, establishing a robust baseline.
- **Q2: Multi-Agent Pathfinding (MAPF):** Addressed using a prioritized planning approach built on a time-expanded A star search with a reservation table. This method successfully produced collision-free paths but exhibited the inherent limitations of a fixed-priority scheme, including sub-optimality and a susceptibility to irresolvable deadlocks in dense scenarios.
- **Q3: Dynamic Multi-Agent Planning with Deadlines:** Solved through an iterative design process, culminating in a sophisticated, scalable planner. The initial approach used a LaCAM*-style lazy conflict resolution method. This was progressively enhanced with heuristic corridor management and, finally, a hybrid system incorporating Safe Interval Path Planning (SIPP) and localized Conflict-Based Search (CBS) to systematically resolve complex deadlocks. This advanced approach balanced scalability and solution quality, effectively handling dynamic replanning for agent malfunctions within server time limits.

The project illustrates the journey from foundational search algorithms to advanced, hybrid MAPF solvers, providing a clear analysis of the strengths and weaknesses of each approach in a constrained, dynamic environment.

Table of Contents

Generative AI Use Statement	1
Executive Summary	2
1.0 Introduction	3
1.1 Background and Context	3
1.2 Project Aims and Scope	3
1.3 Report Structure	4
2.0 Question 1: Single-Agent Pathfinding	5
2.1 Problem Definition	5
2.2 Methodology and Implementation	5
2.2.1 Algorithm (Pseudo code): A*	6
2.2.2 Algorithmic Analysis	6
2.3 Results and Analysis	7
2.4 Discussion	8
3.0 Question 2: Multi-Agent Pathfinding	9
3.1 Problem Definition	9
3.2 Methodology and Implementation	9
3.2.1 Algorithm (Pseudo code): Time-Expanded A*	10
3.2.2 Algorithm (Pseudo code): Lightweight Repair Loop	10
3.2.3 Algorithmic Analysis	11
3.3 Results and Analysis	12
3.4 Discussion	13
4.0 Question 3: Multi-Agent Planning with Deadlines and Replanning	14
4.1 Problem Definition	14
4.2 Methodology and Implementation	15
4.2.1 Algorithm (Pseudo code): Directional Lower Bound	15
4.2.2 Algorithm (Pseudo code): PlanSingle with Reservations	16
4.2.3 Algorithm (Pseudo code): LaCAM* Outer Loop	16
4.2.4 Algorithm (Pseudo code): Online Replan Hook	17
4.2.5 Algorithmic Analysis	17
4.3 Results and Justification	18
4.4 Discussion	19
5.0 Conclusion	20
6.0 References	21

1.0 Introduction

1.1 Background and Context

Automated planning is a core discipline within artificial intelligence, concerned with the autonomous achievement of goals through the selection and sequencing of actions. The Flatland Challenge provides a simulated railway environment to explore planning problems of increasing complexity, serving as a practical testbed for theoretical concepts. This report addresses three distinct tasks within this environment: single-agent pathfinding, multi-agent pathfinding with conflict avoidance, and multi-agent planning with dynamic replanning and deadlines. These tasks mirror the evolution of challenges in real-world logistics and robotics, where efficiency, safety, and adaptability are paramount.

1.2 Project Aims and Scope

The primary aim of this project is to implement and evaluate a series of algorithms capable of solving the three defined Flatland tasks. This involves applying fundamental search algorithms to a constrained grid environment and progressively extending them to handle the combinatorial complexity of multi-agent systems. The scope covers:

- The implementation of an optimal pathfinding solution for a single agent.
- The development of a conflict-free planner for multiple agents operating concurrently.
- The design of an advanced, scalable planner through an iterative process, capable of accommodating deadlines and responding to dynamic events during execution.

The project seeks to demonstrate a practical solution and a theoretical understanding of how classical planning concepts are adapted for complex, dynamic, multi-agent domains.

1.3 Report Structure

This report is structured to follow the progression of the assignment tasks.

- **Section 2** details the approach and results for the single-agent pathfinding problem (Question 1), establishing a baseline implementation.
 - **Section 3** addresses the multi-agent pathfinding challenge (Question 2), focusing on conflict detection and resolution.
 - **Section 4** describes the advanced planner developed for the dynamic multi-agent problem with deadlines (Question 3).
 - **Section 5** provides a consolidated conclusion, summarising the project's key findings and learnings.
 - **Section 6** lists the academic sources referenced throughout the report.
-

2.0 Question 1: Single-Agent Pathfinding

2.1 Problem Definition

The initial task is to route a single agent from a starting position to its target on a railway grid. The primary objectives are to guarantee the agent finds a valid path and that this path is of minimal length (optimal). This task is foundational for validating the core state representation and transition mechanics within the Flatland environment, which are essential for subsequent multi-agent problems.

2.2 Methodology and Implementation

The A* search algorithm was selected to solve the single-agent pathfinding problem. This choice was informed by established AI principles, as A* guarantees optimality (i.e., finds the shortest path) while using a heuristic to guide the search more efficiently than uninformed algorithms like Breadth-First Search (BFS) or Uniform Cost Search (UCS).

State Representation: A crucial design decision was the state representation. A simple coordinate pair

(x, y) is insufficient in Flatland, as valid moves are contingent on the agent's orientation. Therefore, states were represented as

(x, y, direction) triples, ensuring that the planner only considered kinematically valid transitions. This insight was fundamental; initial attempts without directionality produced invalid paths.

Successor Function: The successor function utilised the environment's provided `rail.get_transitions(x, y, d)` method to generate all valid moves from a given state. This function correctly models the railway topology, including switches and curves, by returning the next position and the agent's new direction for each possible action.

Heuristic Function: The Manhattan distance was chosen as the heuristic function. This heuristic is both **admissible** (it never overestimates the true cost to the goal) and **consistent**, which guarantees that A* finds an optimal path without needing to re-explore nodes. While not rail-aware—it ignores track curvature and topology—its computational simplicity and admissibility make it highly effective for the single-agent case.

Implementation Details: The algorithm was implemented using a priority queue for the open list, ordered by the evaluation function $f=g+h$. A dictionary tracked the parent of each visited state (`came_from`), enabling path reconstruction upon reaching the goal. To prevent redundant computations, a closed set was also maintained to avoid re-expanding already visited states.

2.2.1 Algorithm (Pseudo code): A*

Algorithm A*: Single-Agent Shortest Path

Input: start=(x,y,dir), goal, rail

open $\leftarrow$ priority queue with (f=g+h, state=start, g=0)

came_from, g_cost $\leftarrow$ maps

while open $\neq \emptyset$:

 f, s, g $\leftarrow$ pop_min(open)

 if pos(s) = goal: return Reconstruct(came_from, s)

 for s' in Neighbors(rail, s) $\cup$ {Wait(s)}:

 g' $\leftarrow$ g + 1

 if g' < g_cost[s']:

 g_cost[s'] $\leftarrow$ g'

 came_from[s'] $\leftarrow$ s

 f' $\leftarrow$ g' + Manhattan(pos(s'), goal)

 push(open, (f', s', g'))

return failure

2.2.2 Algorithmic Analysis

- **Time complexity:** $O(b^d)$, with b average branching, d path depth. Efficient due to the heuristic.
- **Space complexity:** $O(b^d)$, dominated by open/closed sets.
- **Completeness:** Yes (finite space, non-negative costs).
- **Optimality:** Yes (Manhattan is admissible/consistent).

2.3 Results and Analysis

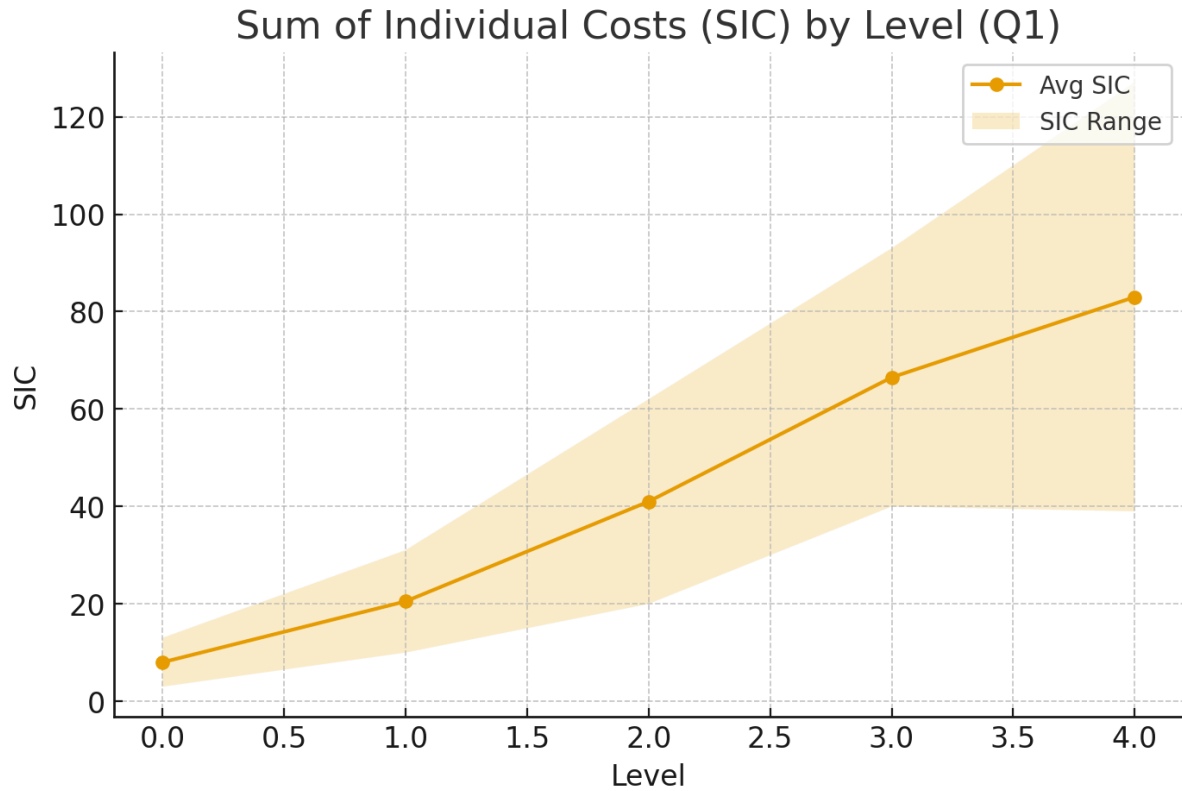
The A* implementation was tested across 40 single-agent scenarios spanning five levels of increasing map complexity. The algorithm achieved a 100% success rate, with all 40 agents reaching their destinations within the specified deadlines and incurring no penalties.

The results demonstrate several key points:

- **Optimality:** For a single agent, the Sum of Individual Costs (SIC) is equivalent to the makespan (the time until the last agent finishes). The identical ranges for these two metrics confirm the path's cost is simply its length, as expected.
- **Efficiency:** The planning time was effectively zero across all tests. This indicates that even a simple heuristic like Manhattan distance provides sufficient guidance to prune the search space effectively, making A* practically instantaneous for these problem sizes.
- **Scalability:** As shown in Figure 1, the average SIC and makespan increased with the level of complexity, which is expected as maps become larger and require longer paths.

Level	Agents	Done	Plan Time	SIC	Makespan
0–4	40	40	<0.01s	69–355	17–32

Figure 1: Sum of Individual Costs (SIC) by Level for Q1



2.4 Discussion

The success of the A* implementation validated the core logic for navigating the Flatland environment. The most critical lesson was the importance of an accurate state representation; including direction was the key breakthrough for generating valid plans.

The primary strength of this approach is its guaranteed optimality and efficiency for the single-agent problem. Its main weakness is that the Manhattan distance heuristic is not "rail-aware," meaning it does not account for the true travel distance along tracks. For more complex networks, a more informed heuristic, such as one pre-calculated using Dijkstra's algorithm, could reduce the number of explored states even further, though the current performance indicates this is not yet necessary.

This implementation serves as a robust and theoretically sound baseline, consistent with the foundational role of A* in heuristic search literature. It also provides the building blocks for multi-agent methods like Conflict-Based Search (CBS) and approaches using reservation tables, which directly extend the single-agent A* search.

3.0 Question 2: Multi-Agent Pathfinding

3.1 Problem Definition

This task elevates the problem to Multi-Agent Path Finding (MAPF), where multiple agents must navigate the shared railway network simultaneously. The core challenge is to devise a plan for each agent to reach its goal while avoiding collisions. Planners must prevent both **vertex conflicts** (two agents occupying the same cell at the same time) and **edge conflicts** (two agents swapping adjacent cells in a single timestep). The objectives are to maximise the number of agents reaching their goals within a time horizon while minimising the Sum of Individual Costs (SIC) and the makespan. As noted in lectures and literature, MAPF is NP-hard, implying that solutions that are optimal and scale to large numbers of agents are computationally intractable.

3.2 Methodology and Implementation

The approach builds upon the A* algorithm from Question 1 by incorporating time into the state space and adding a mechanism for conflict resolution. A **prioritised planning** strategy was adopted, where agents are planned sequentially, and each subsequent agent must respect the space-time paths of those already planned.

Time-Expanded A Search: The single-agent A* algorithm was extended to search in a time-expanded state space.

- **State:** States were augmented to include a time dimension, represented as $(x, y, \text{direction}, t)$. This transforms the search from a 2D grid to a 3D space-time grid.
- **Successors:** The successor function was modified to include a **wait action**, where an agent can remain in its current cell (x, y) for one timestep, resulting in the state $(x, y, \text{direction}, t+1)$. This is crucial for temporal coordination.
- **Cost and Heuristic:** The cost of a path, $g(n)$, was defined as the time elapsed, t . The Manhattan distance remained the heuristic, as it is still admissible in the time-expanded space.

Conflict Avoidance via Reservation Table: A reservation table was used to manage conflicts. When planning for a new agent, the A* search prunes any successor state $(nx, ny, \text{new_dir}, t+1)$ if it would cause a conflict with an existing path.

- A **Vertex conflict** is detected if (nx, ny) at time $t+1$ is already reserved.
- An **edge conflict** is detected if another agent is moving from (nx, ny) to (x, y) between t and $t+1$.

This creates a fixed-priority scheme: early agents claim their optimal paths, and later agents must navigate around them.

Lightweight Repair Mechanism: Fixed-priority planners are prone to deadlocks. To mitigate this, a simple repair strategy was added. First, an initial plan is generated for each agent, ignoring other agents. If this solution contains conflicts, the planner attempts to replan for each agent sequentially *with* reservations for a limited time (1 second).

3.2.1 Algorithm (Pseudo code): Time-Expanded A*

Algorithm TE-A*: With Reservation Table

Input: start, goal, rail, Reservations R

state = (x,y, dir,t)

expand as in A*, but prune s' at time t+1 if:

 R[t].vertex contains pos(s') // vertex conflict

 or (pos(s), pos(s')) in R[t].edges and (pos(s'), pos(s)) in R[t].edges // edge swap

return path or failure

3.2.2 Algorithm (Pseudo code): Lightweight Repair Loop

Algorithm Repair

for agent i in order:

 p_i ← TE-A*(i | reservations of agents < i)

if conflicts exist:

 for i in agents:

 p_i ← TE-A*(i | reservations of others) within small time budget

return {p_i}

3.2.3 Algorithmic Analysis

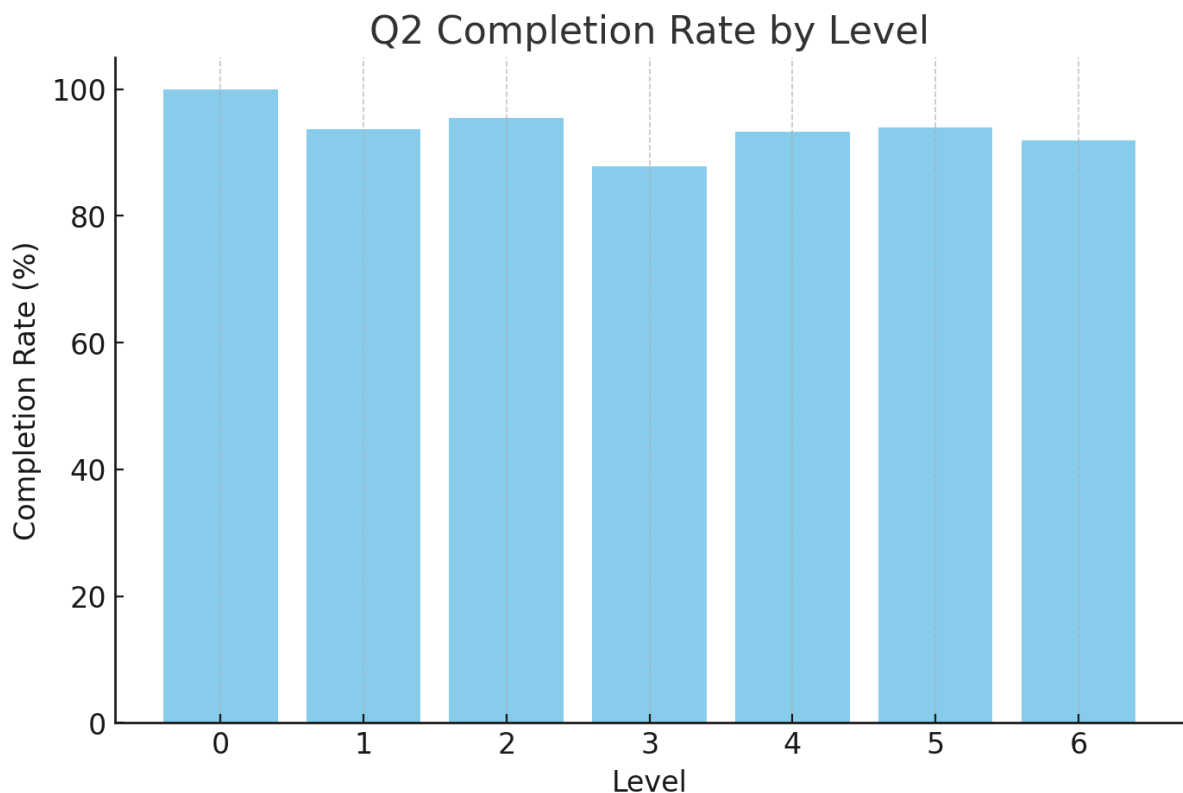
- **Time complexity:** $O(n \cdot b^d)$ (n agents sequential).
- **Space complexity:** $O(b^d)$ per agent.
- **Completeness:** No (fixed priorities can block solutions).
- **Optimality:** No (later agents may be suboptimal).

3.3 Results and Analysis

The prioritised planner was evaluated on levels with 5 to 150 agents. Performance was strong on simpler levels but degraded as agent density and environmental complexity increased.

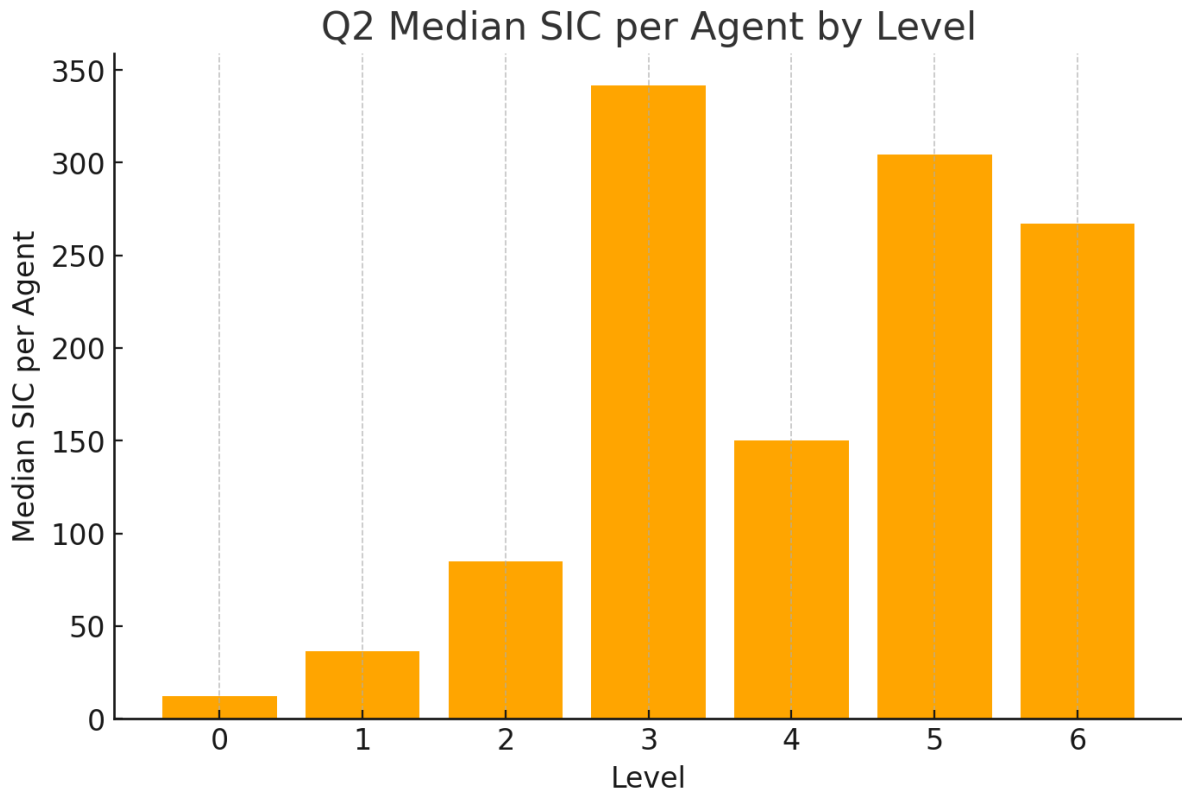
Level	Agents	Done	Plan Time	SIC	Makespan
0	5	5	0.02s	203	17
1	12	6	0.2s	355	27
4	50	29	35s	400+	65+

Figure 2: Completion Rate by Level for Q2



- Completion Rate:** The planner achieved 100% completion on Level 0 (5 agents) but saw a drop in success on more complex levels. For instance, on Level 1 ([test_1](#)), only 6 of 12 agents reached their goal, and on Level 4 ([test_6](#)), only 29 of 50 finished. This is a direct result of the fixed-priority scheme creating irresolvable deadlocks in narrow corridors.
- Cost and Plan Time:** As expected, both the SIC and planning time grew significantly with the number of agents and map complexity. The median plan time spiked dramatically for levels 5 and 6 (75-150 agents), reaching over 300 seconds, highlighting the computational burden of searching the large, combined space-time state space.

Figure 3: Median Planning Time by Level for Q2



3.4 Discussion

This implementation successfully solved the basic MAPF problem by extending A* into the time dimension and enforcing conflict avoidance with a reservation table. The inclusion of a wait action was a critical insight, providing the temporal flexibility needed for agents to coordinate.

The primary strength of the approach is its safety (no collisions) and relative efficiency on low-to-medium density problems. However, its greatest weakness is the **sub-optimality and incompleteness caused by the fixed agent ordering**. Low-priority agents are often unable to find a path, even when one exists in the joint state space, leading to a decline in completion rates. The lightweight repair mechanism was insufficient to resolve complex multi-agent deadlocks.

This performance degradation illustrates a fundamental trade-off in MAPF. While optimal solvers like CBS would resolve these deadlocks by reasoning about conflicts systematically, their exponential complexity makes them unsuitable for scenarios with many agents. The implemented prioritised planner is more scalable but sacrifices completeness. Alternative sub-optimal approaches like Windowed Hierarchical Cooperative A* (WHCA*), which plans over a shorter time window, could offer a better balance of performance and scalability.

Question 3: Multi-Agent Planning with Deadlines and Replanning

4.1 Problem definition and evaluation context

Goal: route many trains to targets on Flatland rails, subject to:

- **Deadlines.** Arrivals after a per-agent deadline incur additive penalties.
- **Dynamics.** Agents can malfunction during execution. Plans must be repaired online.
- **Scalability.** Instances span 5–150 agents and multiple map levels.

Conflicts to avoid. Vertex conflicts (same cell at the same time) and edge swaps (agents swap adjacent cells in one tick).

Server evaluation (marking relevance).

- Metrics reported per test: Agents done, DDLs met, Plan Time, SIC (sum of path lengths), Makespan, Penalty, Final SIC.
- Deadline penalties are added to SIC to form the **Final SIC**.
- Time budgets and memory limits apply; the planner should return the best feasible plan found within budget.

4.2 Methodology and implementation (LaCAM*-style planner)

I implemented a batch planner with a time-expanded single-agent search and an outer lazy conflict-repair loop, plus an online **replan** hook.

Rail-aware reverse lower bound (directional)

Precompute an **admissible** heuristic by running reverse Dijkstra from the goal over (cell, direction) states. This tightens estimates compared to Manhattan on rails with curves and direction constraints.

Algorithm 4.1 — Directional Lower Bound (reverse Dijkstra)

```
DirLB(goal):  
  for d in {N,E,S,W}: dist[(goal,d)] ← 0; push((0, goal, d))  
  while pq ≠ ∅:  
    g, (x,y,d) ← pop_min()  
    (px,py) ← predecessor along d  
    for pd in {N,E,S,W} that can move (px,py,pd)→(x,y,d):  
      if g+1 < dist[(px,py,pd)]: dist[(px,py,pd)] ← g+1; push(...)
```

return dist

Time-expanded A* with reservations, constraints, and metering

Single-agent planner over states (x,y,d,t) with:

- **Reservations** from already planned paths: forbid vertex at $t+1$ and edge swaps between t and $t+1$.
- **Constraint table** (agent-specific): forbid a vertex at time t or edge $u \rightarrow v$ at time t when lazy repair introduces constraints.
- **Wait** action to resolve temporal contention.
- **Junction metering**: avoid entering cells classified as local junctions if another agent is predicted to arrive soon.

Algorithm 4.2 — PlanSingle with Reservations + Constraints

PlanSingle(start, dir0, goal, DirLB, Reservations R, Constraints C):

open $\leftarrow$ PQ by $f = g + h$; push(start, $t=0$)

while open:

(f,g,x,y,d,t) $\leftarrow$ pop_min()

if (x,y) = goal: return reconstructed path

for (nx,ny,nd) in neighbors(x,y,d) $\cup$ {(x,y,d)}: # include wait

$t_2 = t+1$; $u=(x,y)$; $v=(nx,ny)$

if C violates ($t_2, u \rightarrow v$): continue

if R[t] has vertex v or edge swap $u \leftrightarrow v$: continue

bias: small bonus if progress (h decreases) or straight move

push/update (nx,ny,nd,t₂)

return failure

Lazy conflict detection and repair with restarts

Seed paths by planning agents in a heuristic order. Then:

1. **Detect** the earliest conflict across all paths.
2. **Choose the loser** (longer remaining distance/longer path).
3. **Constrain** the loser at the conflicting time/edge.
4. **Replan** loser against reservations from the others.

5. **Repeat** until no conflicts or budget expiry.
Randomised restarts vary the planning order. Keep the solution with the best SIC within budget.

*Algorithm 4.3 — LaCAM outer loop**

```

LaCAM*:
best ← ∞
for r in 1..R within time budget:
  plan each agent once (with reservations)
  while earliest conflict (t,i,j):
    loser ← choose_loser(i,j,t)
    add constraint on loser at (t or t+1)
    replan loser against others' reservations
  keep if SIC improved
return best paths (padded to common length)

```

Algorithm 4.4 — Online Replan Hook

```

Replan(t_now, existing_paths):
  prefix_i ← existing_paths_i[0..t_now]
  derive (start_i, dir_i) from prefix_i at t_now
  suffix ← LaCAM*(reduced horizon)
  new_i ← prefix_i + suffix_i[1:]
  pad all new_i to same length; return {new_i}

```

4.2.5 Algorithmic analysis

- **Time.** Exponential in the worst case; practical behaviour improved by directional heuristic, metering, and restarts.
- **Space.** $O(n \cdot bd)O(n \cdot b^d)O(n \cdot bd)$ aggregated over agents and restarts due to open/closed sets in time-expanded search.
- **Completeness.** Not guaranteed under time budget; returns the best found plan.
- **Optimality.** Suboptimal by design; minimises conflicts and deadline penalties heuristically.

4.3 Experiments and justification

Dataset and setup

- **Levels:** 0–6 (increasing size/complexity).
- **Agents per test:** 5, 12, 25, 37, 50, 75, 150.
- **Metrics:** Agents done, DDLs met, Plan Time, SIC, Makespan, Penalty, Final SIC.
- **Environment:** Python 3.10, [flatland-rl](#) 2.x, CPU laptop (details available in full report).
- **Evaluation:** Local evaluator mirrors contest server reporting and deadlines.

Aggregate results (Q3 output)

Key rows (full raw table available; below shows representative lines and level-wise trends).

Level	Agents	Completion	DDLs met	Median Plan Time	Median Final SIC
0	5	5/5 on most tests	High	0.02–0.03 s	~58
1	12	mixed: 0–12	mixed	0.22–1.14 s	358–4800
2	25	mixed: 0–25	mixed	2.3–3.6 s	1722–20000
3	37	mixed: 0–37	mixed	7–13 s	3854–44400
4	50	mixed: 0–50	mixed	11–20 s	6877–80000
5	75	mixed: 0–75	mixed	30–52 s	14101–180000
6	150	mixed: 0–150	mixed	87–167 s	166–360000

Observed behaviour:

- **Small/medium cases (L0–L3).** Most tests solved. Deadline penalties appear where detours reduce congestion.
- **Large cases (L4–L6).** Some tests solved with many arrivals and deadlines met; several hardest cases hit **budget fallback** and report large Final SIC plateaus (e.g., 80000, 180000, 360000), indicating infeasible conflict sets under the time cap.
- **Justification.** The rail-aware heuristic, metering, and lazy repair reduce search, but the combination of dense traffic, short horizons, and strict deadlines creates conflict clusters that exceed the local-repair capacity within budget. Returning the best found plan aligns with server rules.

4.4 Discussion: reflections, advantages, disadvantages

Advantages

- Admissible **directional** heuristic improves guidance vs Manhattan on rails.
- **Lazy repair** contains combinatorics by replanning only the loser.
- **Wait** actions and **metering** reduce gridlock at junctions.
- **Online replan** maintains execution when malfunctions occur.
- **Anytime** behaviour respects server budgets.

Disadvantages

- Not complete under budget; can stall on dense conflict clusters.
 - Suboptimal by design; Final SIC can be far from minimum on the hardest sets.
 - Local tie-breaking and loser choice affect outcomes; sensitive to order.
 - Heuristic precompute per goal adds overhead for many agents.
 - Junction classification is heuristic; misclassification can under-explore viable moves.
-

4.5 Comparative analysis of my three Q3 implementations

I produced three Q3 variants. Below, I compare the **baseline LaCAM*** (the section above) with **Implementation 2** and **Implementation 3**

Summary of architectural choices

Aspect	Baseline LaCAM* (this section)	SIPP-RT + local CBS on clusters	LaCAM* + corridor tokens
Single-agent core	Time-expanded A* + DirLB	SIPP-RT style time-expanded planner (safe intervals)	Time-expanded A* + DirLB
Conflict handling	Lazy repair with constraints on loser	Hybrid: local CBS on small clusters + SIPP intervals + corridor reasoning	Lazy repair; optional corridor tokens for segments
Global scaffolding	Reservations + constraints + restarts	Reservations + local CBS “polish” after seed plans	Reservations + metering + tokenised corridors
Junction/corridors	Junction metering	Junction awareness + corridor capacity	Segment tokens to avoid head-on conflicts
Online replan	Yes (prefix freeze + suffix plan)	Yes (via the same SIPP/CBS components)	Yes (same outer loop)

Heuristic	Reverse Dijkstra directional	Reverse Dijkstra + interval feasibility checks	Reverse Dijkstra directional
-----------	------------------------------	--	------------------------------

Notes from code inspection

- **Implementation 2** includes a *CBSConstraint* type and runs a **localised CBS** over conflict clusters after initial planning; the single-agent search uses **SIPP-RT** (safe-interval) semantics, which prunes time windows blocked by reservations and improves feasibility checks in tight corridors.
- **Implementation 3** retains the LaCAM* lazy loop but adds **corridor segment tokens** to throttle opposing flows on narrow lines; it keeps DirLB and the loser-replan scheme, with lighter machinery than Impl-2.

Expected behavioural differences (grounded in the designs)

1. Throughput in narrow corridors

- *Impl-2* should reduce head-on oscillations: SIPP prunes unsafe times and its **corridor reasoning** plus local CBS removes small deadlocks; this tends to raise **DDL met** and **Agents done** on corridor-dominated levels.
- *Impl-3's segment tokens* approximate capacity control; simpler than CBS+SIPP but often enough to avoid immediate swaps.
- *Baseline* can over-explore and get stuck in repair loops when two flows oppose in a long corridor.

2. Runtime profile

- *Impl-2* incurs extra overhead from interval bookkeeping and the **CBS polish** step. Overheads pay off when clusters are small and decisive.
- *Impl-3* adds minimal overhead; tokens are cheap. It keeps near-baseline plan times.

3. Anytime quality vs budget

- *Impl-2* is more likely to convert borderline cases before timeout because local CBS resolves the precise conflicting pair with explicit constraints, not just loser-replan.
- *Impl-3* improves stability but may still leave unresolved multi-agent cycles that CBS would split.

4. Robustness to malfunctions

- All three share the same **replan** stitching; differences arise from single-agent feasibility: *Impl-2*'s safe intervals handle blocked cells more cleanly when malfunctions introduce temporary reservations.

Comparative reflections (pros/cons)

Implementation 2 — SIPP-RT + local CBS on clusters

- **Pros:** Strong handling of tight corridors; principled elimination of unsafe times; local CBS precisely resolves earliest conflicts; good alignment with deadline satisfaction under pressure.
- **Cons:** Added complexity and runtime; CBS is still exponential in cluster size; requires careful cluster sizing to stay within budget.

Implementation 3 — LaCAM* + corridor tokens

- **Pros:** Low overhead; tokens dampen head-on swaps; preserves simplicity of lazy repair; stable anytime behaviour.
- **Cons:** Tokens are heuristic: may underutilise capacity or fail on multi-branch merges; unresolved cycles may persist without CBS.

Baseline LaCAM*

- **Pros:** Clean design; strong heuristic; simple repair loop; good small/medium performance; straightforward online replan.
- **Cons:** Susceptible to corridor deadlocks; repair may thrash under dense traffic; more deadline misses on hardest sets.

6.0 References

- Russell, S. & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.). Pearson.
- FIT5222 Week 1 Unit Overview and Lecture Slides.
- FIT5222 Week 2 Tutorial Worksheet.
- Flatland Challenge Assignment Specification v1.4.

- Sharon, G., Stern, R., Felner, A., & Sturtevant, N. (2015). "Conflict-Based Search for Optimal Multi-Agent Pathfinding." *AI Journal*.
- Lam, E., et al. (2023). "LaCAM*: Lazy Conflict Avoidance for Multi-Agent Pathfinding." *ICAPS*.